

Technical Analysis of Distributed Autonomous AI Architectures in the Sereeps Computational Environment

The engineering of fully autonomous agents within the Sereeps environment represents a sophisticated challenge in real-time distributed systems, necessitating the development of complex software architectures that manage resource allocation, spatial reasoning, and strategic decision-making. Unlike traditional strategy games that rely on human-in-the-loop interaction, Sereeps functions as a persistent, 24/7 simulation where player-written JavaScript or TypeScript code must execute within a strict central processing unit (CPU) and memory budget.¹ The primary architectural patterns that have emerged in this domain range from modular analyst-supervisor models, such as the JonWinsley "Grey Company" bot, to hierarchical hiveminds like Overmind, and virtualized operating systems such as Sereeps-OS.³ Each paradigm addresses the fundamental problem of how to efficiently translate environmental data into actionable intents while maintaining scalability across hundreds of game rooms.²

The Server-Side Infrastructure and Computational Constraints

To analyze the performance of AI architectures, it is necessary to first understand the server-side environment in which they operate. The Sereeps world is hosted on a distributed network utilizing Node.js (v8.9.3 or v16 in newer iterations), MongoDB, and Redis.² A single game tick is processed in a two-stage lifecycle.² In the first stage, the server calculates player scripts in parallel across multiple quad-core dedicated servers.² This stage involves fetching game object data from MongoDB and loading it into RAM, where the player's script is executed using the Node.js `vm.runInContext` library.² The second stage processes the resulting "intents"—commands such as move, harvest, or transfer—to update the global game state.²

The primary constraint for any architecture is the CPU limit, which is a per-tick allowance based on the player's Global Control Level (GCL). To manage spikes in computation, the engine provides a "Bucket" system that allows players to accumulate up to 10,000 unused CPU cycles.⁸ Furthermore, memory is limited to a 2048 KB JSON-serializable object that persists between ticks.² These constraints necessitate architectures that minimize redundant calculations and optimize memory access patterns.¹⁰

Infrastructure Component	Specification / Technical Detail	Functional Role
Runtime Environment	Node.js (v8.9.3 to v16)	Execution of player-written JavaScript/TypeScript ²
Persistent Database	MongoDB 3	Storage of room states and game object documents ²
Synchronization Layer	Redis 3	Tick synchronization and task queue management ²
Memory Object	2 MB JSON String	Persistent state storage across game ticks ⁹
Execution Context	vm.runInContext	Isolated sandbox for player script execution ²

The Modular Analyst-Supervisor Model: The JonWinsley Architecture

The JonWinsley architecture, often referred to as the "Grey Company," is a highly modular paradigm designed for maximum efficiency, particularly on Shard 3 where CPU resources are severely restricted.⁶ This model rejects the standard tutorial-style if-else loops in favor of a specialized hierarchy involving Architects, Managers, Supervisors, and Analysts.¹² The design philosophy centers on the decoupling of data gathering (Analysts) from goal setting (Managers) and task execution (Supervisors and Minions).³

Functional Roles in the Grey Company Framework

The JonWinsley model utilizes a clear separation of concerns to prevent the "spaghetti code" common in simpler bots. This modularity allows for easier debugging and profiling of specific system components.³

Architects and Room Planning

Architects are responsible for long-term spatial reasoning. This module analyzes the terrain of a room to determine the optimal layout for infrastructure, such as Franchises (remote outposts) and the Headquarters (the central base block).⁶ Because room planning is computationally expensive, Architects generate a plan only once or upon significant environmental changes, converting it into a minimal representation cached in Memory to save CPU on subsequent ticks.⁶ The architecture assumes a 13x13 open area for the initial base, often requiring a specific

naming convention (e.g., {roomName}0) for city identification.³

Managers and Resource Monitoring

Managers act as the tactical decision-makers. They monitor the state of specific resources—such as energy levels in extensions, the health of walls, or the availability of minerals—and create "Tasks" when a need is identified.¹² For instance, if a tower's energy falls below a certain threshold, the Manager generates a refill task.¹² This reactive model ensures that units are only active when there is useful work to be done, minimizing "idle" CPU waste.¹⁰

Supervisors and Minion Delegation

Supervisors serve as the dispatchers of the system. They take the pool of active tasks and delegate them to available units, referred to as "minions".¹² The Supervisor manages priorities, ensuring that critical operations like filling spawns and extensions (necessary for continued unit production) are addressed before lower-priority tasks like upgrading the room controller.¹³

Analysts and Data Caching

Analysts are the most critical component for CPU optimization in the JonWinsley framework. They collate game data (e.g., the position of all dropped energy or the state of all containers) and process it into easily accessible formats for other modules.⁶ By centralizing data gathering, the AI avoids redundant calls to the `room.find()` method, which is a significant source of CPU overhead in the Screeps API.¹⁰

Optimization Strategies for Low-CPU Environments

The JonWinsley bot is specifically tuned for Shard 3, where CPU limits are tight. This is achieved through a "Direct Delivery" model where energy is routed directly from the source to its destination whenever possible, reducing the "double-handling" of resources that occurs when everything is moved to a central storage first.⁶ Furthermore, the bot utilizes a Rust-based WebAssembly (WASM) pathfinder to offload complex A* calculations from the JavaScript main loop, achieving significantly higher performance than the default engine pathfinder.⁶

Optimization Technique	Implementation Detail	Target Efficiency
WASM Pathfinding	Rust-compiled A* / Dijkstra algorithms	CPU Efficiency ⁶
Analyst Caching	Single-tick data collation to avoid <code>room.find</code>	CPU Efficiency ¹⁰

Minimal Representation	Caching base plans as compact strings in Memory	Memory Efficiency ⁶
Direct Delivery	Routing energy source-to-sink without storage hub	Energy Efficiency ¹⁴
Dynamic Body Logic	Sizing creeps based on distance and workload	Energy/Spawn Efficiency ¹³

Hierarchical Hivemind Architectures: The Overmind Framework

The Overmind bot, developed by Ben Bartlett, utilizes a deeply hierarchical, object-oriented (OO) architecture that organizes game logic into an object tree.⁴ This framework is designed for high-level abstraction, allowing the player to manage thousands of units ("Zerg") through a centralized "Hivemind" that coordinates multiple "Colonies".¹

The Structural Hierarchy of Overmind

The Overmind architecture is built around several core classes that manage the AI's lifecycle. Each tick, the bot iterates through a specific three-phase cycle: build(), init(), and run().⁴

Colonies and Hive Clusters

A "Colony" represents the presence of the AI in a single room. It is a top-level entity that instantiates sub-components such as hive clusters (groups of structures like spawns and labs) and logistics networks.⁴ The Colony serves as the root for all localized decision-making, including resource distribution and defense.⁴

Directives and the Overseer

"Directives" are specific, mission-oriented goals placed in the game world, often signaled by flags.⁴ For example, a "Mining Directive" tells the AI to establish a remote harvesting operation, while a "Combat Directive" coordinates an attack.⁴ The "Overseer" component dynamically adds or removes these directives based on intelligence gathered from the environment.⁴

Overlords and Zerg

"Overlords" are the middle managers of the hierarchy. Each Overlord is responsible for a specific directive and manages a group of "Zerg" (a wrapper class for creeps) to fulfill it.⁴ This abstraction allows the AI to give high-level commands ("defend this room") while the Overlord

handles the low-level tactics of unit positioning and combat.⁴

V8 Engine Optimization: The Refresh Mechanism

A major challenge with deeply nested OO architectures in JavaScript is the high cost of class instantiation and the resulting pressure on the V8 garbage collector.⁴ Overmind addresses this through a "refresh" phase. Instead of re-instantiating the entire object tree every tick, the bot keeps the class instances for Colonies, Overlords, and Directives alive across ticks.⁴ Every 20 ticks, it runs a full build() to refresh the hierarchy; on all other ticks, it runs a refresh() method that simply updates the internal references to game objects while maintaining the existing script state.⁴ This "soft update" significantly reduces the CPU overhead associated with garbage collection.⁴

The Assimilator and Collective Intelligence

A unique feature of Overmind is the "Assimilator," a mechanism that allows multiple players running the Overmind script to coordinate as a single entity.⁴ The Assimilator uses SHA-256 cryptographic hashes to verify that other players are running a compatible version of the script.⁴ Once verified, these players can share a "master ledger" of directives, allowing their creeps to assist each other with resource transfers, defense, and territory expansion.⁴ This turns individual bots into a true "Hivemind" where the collective intelligence of the swarm overrides individual player boundaries.⁴

Operating System Paradigms: The Screeps-OS Model

A more radical departure from standard script design is the "Operating System" (OS) model. In this architecture, the player's code is organized as a set of virtualized processes and threads managed by a Kernel and a Scheduler.⁵ This approach provides a layer of abstraction that allows the AI to manage CPU resources more effectively than a standard main loop.⁵

Core Components of a Screeps-OS

The OS model mimics the structure of a real operating system to manage the complexity of late-game execution.⁵

The Kernel and Process Table

The Kernel is the central orchestrator that keeps the entire system together. It provides an API for adding and removing processes and manages the "Process Table," which tracks every active module's state, ID, and priority.⁵ A "Process Registry" is often used to organize available process types, making it easier to spawn different behaviors based on the server environment (e.g., private server vs. public shard).⁵

The Scheduler and CPU Management

The Scheduler is responsible for determining the execution order of processes. It can implement various algorithms such as Priority-Based Scheduling, First-Come-First-Served (FCFS), or Round Robin.⁵ The primary advantage of an OS scheduler in Screeps is its ability to handle "CPU Starvation".⁵ If the player's CPU bucket is low, the scheduler can pause low-priority processes (like market analysis) to ensure that mission-critical processes (like creep movement and defense) have enough resources to run.⁵

System Calls and Generators

In advanced OS implementations like ScreepsOS (SOS), processes utilize JavaScript generators (function*) to enable context switching.¹⁹ A process can "yield" a "System Call" (syscall) to the kernel, which then pauses the process and performs a specific action.¹⁹ Common syscalls include Sleep(n) to pause for a number of ticks, Fork(priority, class) to start a child process, and Yield() to allow the scheduler to interrupt the current process and resume it later in the same tick.¹⁹

Serializability and Memory Management

A key feature of Screeps-OS is its focus on serializability. To survive the server's tick-based execution cycle, the entire state of the OS—including the process table and the stack of each generator—must be restorable from the JSON Memory object.¹⁹ This allows the bot to "stretch" a single complex calculation over multiple ticks without losing its place, effectively removing the 500ms execution limit as a hard barrier for complex tasks.⁸

OS Component	Functional Implementation	Behavioral Advantage
Kernel	API for process instantiation and state tracking	Centralized management of complex AI modules ⁵
Scheduler	Priority-based execution logic (e.g., Round Robin)	Prevents critical process starvation during CPU spikes ⁵
Process	Modular logic container (Generator-based)	Allows for non-blocking, multi-tick calculations ¹⁹
File System	Abstraction of RawMemory into segments/cache	Simplifies data access and improves memory

		efficiency ⁵
Syscalls	Yield, Sleep, Fork, Kill, Inject	Provides fine-grained control over execution flow ¹⁹

Spatial Reasoning and Automated Base Planning Algorithms

Regardless of the high-level architecture (Modular, Hivemind, or OS), every advanced Screamers bot must solve the problem of automated room planning. Manually placing structures is untenable as an AI expands to dozens of rooms.²⁰ Bots utilize a variety of spatial algorithms to determine the optimal layout for spawns, extensions, towers, and ramparts.²⁰

Distance Transform for Open Space Identification

The "Distance Transform" algorithm is the primary tool for identifying building sites. It calculates the distance from every walkable tile in a room to the nearest wall.²⁰ For a bot requiring a large open area (e.g., an 11x11 square for a bunker), it searches for tiles with a distance transform value of 5 or greater.²⁰ This ensures the base is placed in a clear area, minimizing the risk of "dead ends" where creepers can get stuck.²⁰

Floodfill for Accessibility Mapping

Once potential sites are found, a "Floodfill" algorithm is used to categorize tiles by their accessibility.²⁰ This helps the bot determine which tiles are closest to the room controller or energy sources. The bot then allocates these tiles for specific structures, prioritizing those that require frequent access, like extensions and storage.²⁰

Minimum Cut for Defensive Perimeter Design

The "Minimum Cut" (Min-Cut) algorithm is used to determine the most efficient placement of ramparts and walls.²⁰ By treating the room as a flow network where "flow" represents potential enemy paths, the bot finds the smallest number of tiles that must be blocked to fully isolate the base from the room's exits.²⁰ This minimizes the number of ramparts that need to be maintained and repaired, significantly reducing energy and CPU costs over time.²⁰

The Bunker vs. Franchise Layout Comparison

There are two dominant schools of thought regarding base design:

1. **Bunkers:** A highly compact, square layout (often 11x11) where all critical structures are enclosed within a single perimeter.¹⁶ Bunkers are easier to defend and optimize for "fast

filling" (using a dedicated creep to quickly transfer energy from storage to extensions).¹⁶

2. **Franchise/Modular Layouts:** Used by JonWinsley, this involves breaking the room into functional blocks (Franchises at sources, Headquarters at the controller) connected by roads.⁶ This design is more CPU-efficient for movement as it minimizes the distance between energy harvesting and consumption points.⁶

Base Feature	Bunker Design (Overmind Style)	Franchise Design (JonWinsley Style)
Primary Goal	Defensibility and Logistics ¹⁶	CPU and Energy Efficiency ⁶
Footprint	Compact 11x11 or 13x13 ²²	Distributed across room modules ⁶
Energy Flow	Central Storage Hub ¹⁴	Direct from Source to Sink ⁶
Defense	Minimum Cut Perimeter ²⁰	Modular Ramparts and Towers ³
Scaling	Uniform across all rooms ¹⁶	Adapted to room terrain ⁶

Economic and Industrial Automation

A successful Screenshot AI must manage a complex industrial economy involving energy, minerals, commodities, and power.³

Lab Reactions and Power Boosting

Advanced bots like JonathanSafer's bot automate the entire mineral processing chain. This includes scouting for minerals, mining them with specialized creeps, and transporting them to Labs for chemical reactions.³ The bot must determine which "boosts" are needed for its military units (e.g., XUH2O for attack or XLHO2 for healing), coordinate the delivery of precursors via the Market or Terminal, and manage the reaction timing to maximize throughput.³

Market and Terminal Management

The Market system allows bots to trade resources for "Credits," the global game currency.³ Architectures typically include a "Market Manager" or "Ferry" role that monitors price fluctuations and automatically sells excess energy or minerals while buying needed components for boosts or construction.³ This requires the bot to manage "Terminal" structures,

which can send resources across rooms at a cost of energy.⁷

Power Creeps and Power Mining

Late-game players utilize "Power Creeps"—hero units that do not have a standard lifespan and can perform powerful "Ops" such as boosting spawn speed or increasing tower damage.³ Managing power creeps requires a specialized infrastructure to mine "Power" from highway rooms and process it in a "Power Spawn".³ Bots must coordinate long-range military escorts for power miners to protect them from "Harassers" and other players.³

Military Operations and Unit Coordination

Combat in Screeps requires sophisticated coordination between multiple units with different roles. Unlike economic units, military creeps must act in concert to be effective.³

The Quad Formation

One of the most effective military patterns is the "Quad," a group of four ranged units that move in a 2x2 square.³ Quads are highly resilient because they can focus fire on a single target while using a "shared healing" logic where each unit heals the most damaged member of the group.³ Implementing Quads requires advanced pathfinding that accounts for the 2x2 footprint of the group, as they cannot pass through 1-tile wide corridors.²⁰

Specialized Military Roles

Architectures typically define a set of core military roles to handle different combat scenarios³:

- **Breakers:** Units designed to destroy enemy structures, often paired with Medics.³
- **Medics:** Specialized healing units that stay adjacent to combat units.³
- **Defenders:** Units optimized for killing NPC invaders or small enemy scouts.³
- **Harassers:** Ranged units used to disrupt enemy remote mining operations in highways.³
- **Robbers:** Specialized carry-heavy units used to steal resources from abandoned or poorly defended enemy rooms.³

Flag-Based Commanding and Coordination

Coordination is often achieved through a "Flagging" system. A player (or the AI itself) places flags in the game world to signal missions.³

Flag Type	Functional Trigger	Mission Objective
claim	Spawns a claimer and a	Expanding into a new room

	spawn-builder	³
plan	Triggers the Architect to layout structures	Automating base construction ³
break	Dispatches Breakers and Medics	Destroying specific enemy structures ³
quadRally	Assembles four units into a Quad	Coordinated multi-unit assault ³
powerMine	Dispatches mining and hauling units	Collecting power from highway rooms ³

Efficiency and Optimization: The Competitive Edge

The ultimate performance of a Screenshot AI is measured by its efficiency. Community wisdom identifies three primary targets: CPU efficiency, Energy efficiency, and Spawn efficiency.²³

The Efficiency Target Triangle

These targets are often mutually exclusive.²³ For example, "locking" a creep to perform only one "intent" per tick (e.g., move OR transfer) saves CPU but reduces "Game Efficiency" because it takes more ticks to complete a task.¹⁰ Conversely, calculating an exact path every tick maximizes "Energy Efficiency" (by ensuring the shortest path) but consumes significant CPU.¹⁰

Caching and Intent Management

To optimize CPU usage, advanced bots implement extensive caching.¹⁰ This includes:

- **Path Caching:** Storing frequently used paths (e.g., from a source to the storage) in Memory so they do not need to be recalculated.⁶
- **Target Caching:** Ensuring that a creep remembers its current task (e.g., "fill extension X") across ticks so it doesn't run its target-selection logic every time.¹¹
- **Flowfields:** Instead of individual pathfinding, a "Flowfield" calculates a direction vector for every tile in a room towards a single goal. Creeps then simply "look up" their next move based on their current tile, which is significantly faster than A*.¹⁰

Tick Offsetting and Load Balancing

Bots often use "Tick Offsetting" to spread CPU load.¹¹ Instead of running every check every tick, the AI uses modular arithmetic: $\text{Game.time} \% 5$ for frequent tasks, $\text{Game.time} \% 20$ for less

frequent tasks, and Game.time % 100 for long-term strategic analysis.¹¹ This prevents "CPU spikes" where the bot tries to do everything at once and exceeds its limit.⁸

The Future of Sereeps Architectures: LLMs and Automated Search

A significant emerging trend is the application of Large Language Models (LLMs) to the design of Sereeps architectures.²⁵ Research into "Design Principle Transfer" (LAPT) suggests that LLMs can be used to reason about architectural components and prune the search space for optimal AI configurations.²⁵ By representing bot architectures as "Linguistic Descriptions," developers can use LLM-assisted frameworks to automatically adapt their bots to new server rules or combat scenarios.²⁵ This "Fractal Intelligence" approach, where a "Cognitive OS" instantiates specialized sub-minds for specific tasks, represents the next level of evolution beyond standard procedural or object-oriented bots.²⁶

Synthesis and Conclusion

The architecture of a Sereeps bot is a complex interplay of spatial reasoning, resource management, and computational optimization. The JonWinsley modular analyst-supervisor model provides a high degree of transparency and efficiency, making it highly effective in resource-constrained environments like Shard 3. Its focus on specialized roles and Rust-powered pathfinding demonstrates a commitment to technical precision. In contrast, the Overmind framework showcases the power of hierarchical object-oriented design and collective intelligence, allowing for emergent behaviors and cross-player cooperation that would be difficult to implement in flatter architectures.

The Sereeps-OS paradigm represents the most advanced form of resource management, virtualizing the game tick itself to provide robust CPU bucket management and multi-tick task execution. By treating game logic as scheduled processes, OS-style bots ensure that critical defense and economic operations are never starved of resources. When combined with automated base planning algorithms like Min-Cut and Distance Transform, these architectures enable a level of autonomy that allows a player to thrive in a persistent, competitive world with minimal manual intervention. As the field moves toward LLM-assisted architecture search and automated design principle transfer, the boundary between human-written scripts and evolved artificial intelligence will continue to blur, setting the stage for even more sophisticated autonomous systems in the years to come.

Works cited

1. Overmind - Ben Bartlett, accessed February 24, 2026, <https://bencbartlett.com/projects/overmind/>
2. Server-side architecture overview | Sereeps Documentation, accessed February 24, 2026, <https://docs.sereeps.com/architecture.html>

3. JonathanSafer/screeps: Screeps AI - GitHub, accessed February 24, 2026, <https://github.com/JonathanSafer/screeps>
4. Screeps #6: Verifiably refreshed | Ben Bartlett, accessed February 24, 2026, <https://benbartlett.com/blog/screeps-6-verifiably-refreshed/>
5. Operating System - Screeps Wiki, accessed February 24, 2026, https://wiki.screepspl.us/Operating_System/
6. Screeps #15: Room Planning | Field Journal, accessed February 24, 2026, <https://www.jonwinsley.com/notes/screeps-room-planning>
7. Mirroar/hivemind: Fully automated open source AI core for ... - GitHub, accessed February 24, 2026, <https://github.com/Mirroar/hivemind>
8. To Code Operating Systems or Not : r/screeps - Reddit, accessed February 24, 2026, https://www.reddit.com/r/screeps/comments/790jh0/to_code_operating_systems_or_not/
9. Community Communication - Screeps Wiki, accessed February 24, 2026, https://wiki.screepspl.us/Community_Communication/
10. CPU Optimization | Screeps Forum, accessed February 24, 2026, <https://screeps.com/forum/topic/1614/cpu-optimization>
11. Need some help improving my CPU usage | Screeps Forum, accessed February 24, 2026, <https://screeps.com/forum/topic/2381/need-some-help-improving-my-cpu-usage>
12. Screeps #1: The Game Plan | Field Journal, accessed February 24, 2026, <https://jonwinsley.com/notes/screeps-game-plan>
13. Screeps #18: Spawn Uptime - Field Journal, accessed February 24, 2026, <https://jonwinsley.com/notes/screeps-spawn-uptime>
14. Screeps #10: Questioning Everything | Field Journal, accessed February 24, 2026, <https://jonwinsley.com/notes/screeps-questioning-everything>
15. Screeps #22: For Science | Field Journal, accessed February 24, 2026, <https://jonwinsley.com/notes/screeps-for-science>
16. Screeps #5: Evolution | Ben Bartlett, accessed February 24, 2026, <https://benbartlett.com/blog/screeps-5-evolution/>
17. (Operating System) Week 6 - Process Management (Scheduling) | PDF - Scribd, accessed February 24, 2026, <https://www.scribd.com/document/850202404/Operating-System-Week-6-Process-Management-Scheduling>
18. CSI-503 - 2. Processor Management | PDF - Slideshare, accessed February 24, 2026, <https://www.slideshare.net/slideshow/csi503-2-processor-management/250938951>
19. screepers/ScreepsOS: Screeps/Serializable Operation ... - GitHub, accessed February 24, 2026, <https://github.com/screepers/ScreepsOS>
20. Automating Base Planning in Screeps – A Step-by-Step Guide, accessed February 24, 2026, <https://sy-harabi.github.io/Automating-base-planning-in-screeps/>
21. Screeps: First Steps in Automating Room Planning | by Jerrold Moore | Medium,

- accessed February 24, 2026,
<https://medium.com/@jerroydmoore/screeps-first-steps-in-automating-room-planning-6fcb2ec89c1a>
22. Automated Base Planning | Screeps Tutorial - YouTube, accessed February 24, 2026, <https://www.youtube.com/watch?v=YcruUDbqa7E>
 23. Optimization - Screeps Wiki, accessed February 24, 2026, <https://wiki.screepspl.us/Optimization/>
 24. screeps/doc/Manual.md at master - GitHub, accessed February 24, 2026, <https://github.com/TooAngel/screeps/blob/master/doc/Manual.md>
 25. Design Principle Transfer in Neural Architecture Search via Large Language Models, accessed February 24, 2026, <https://ojs.aaai.org/index.php/AAAI/article/view/34463/36618>
 26. A New Scaling Law for AI: From Fractal Intelligence to a Hive Mind of Hive Minds – A Paradigm Shift in AGI Design : r/PromptEngineering - Reddit, accessed February 24, 2026, https://www.reddit.com/r/PromptEngineering/comments/1lsslos/a_new_scaling_law_for_ai_from_fractal/